

Parametric and non-parametric models

Parametric model:

- Hypotheses are defined by parameters

- Linear Regression
- Logistic Regression
- Perceptron
- Neural Networks (we will see them)

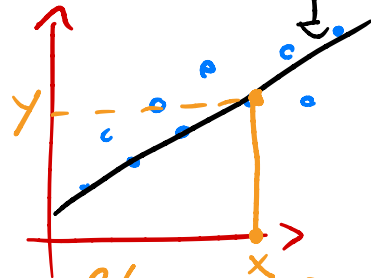
WHY? → BECAUSE THE RESULT OF THE HYPOTHESIS DEPENDS ON THE PARAMETERS

$f: \mathbb{R} \rightarrow \mathbb{R}, h(x; w_1, w_0) = w_1 x + w_0$

HYPOTHESIS FUNCTION

the parameters

SQUARED ERROR MINIMIZATION



Non-parametric model:

- Hypothesis induced by dataset

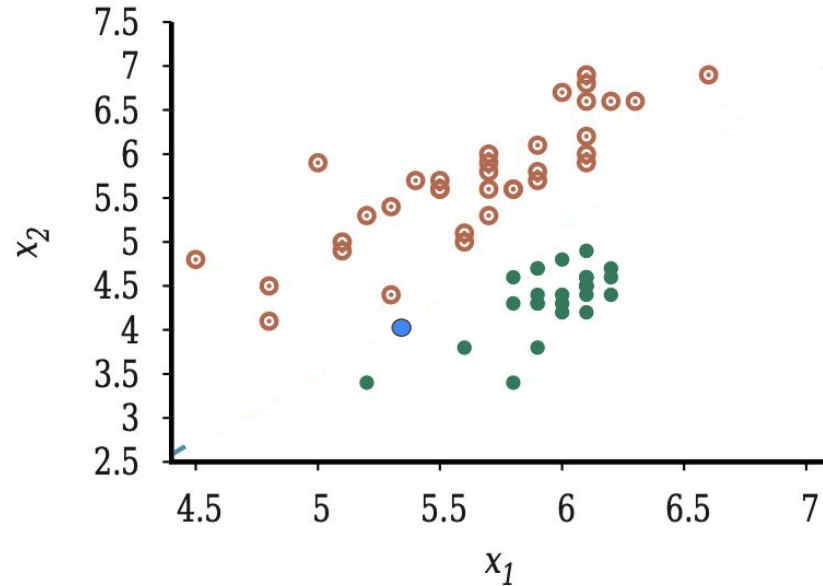
EVERY DATASET INCLUDES AN HYPOTHESIS

EXAMPLE: $h(x; w_1, w_0) = 1.7x + 0.1$
Y IS GIVEN BY w_1, w_0 FOR A SPECIFIC POINT. When you have that function, you can throw away the dataset

Learning with non-parametric model a.k.a. Instance-based Learning

k Nearest Neighbors (kNN) DISTANCE-BASED APPROACH

Binary Classification example: How to classify the blue instance?



How do you classify that blue dot? I simply check if it is similar to the orange one or to the green one. Similar = closest one to **distance** approach.

k Nearest Neighbors (kNN)

Intuition: close-by points are likely to ~~have same~~ have same labels

Close-by: need for distance measure

THEY WILL HAVE SIMI-
LAR FEATURES!
EX.: CLASSIFYING
PEOPLE WHO MOSTLY
LIKE A SPECIFIC MOVIE

$$\rho : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$$

Typically, Euclidean distance:

$$\rho(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\| = \sqrt{\sum_{i=1}^d (x_i - x'_i)^2}$$

kNN Binary Classification

lets see the usage of kNN for binary classification



it is a sequence:
"it can be multiple instances of the same object"

N.B.: IT'S NOT A SET

Consider training examples: $S = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$

Given instance $\mathbf{x} \in \mathcal{X}$ and number k , **THE BLUE DOT; k : HOW MANY INSTANCES FROM S TO COMPARE WITH**

Order all examples from S based on distance function $\rho(\mathbf{x}, \mathbf{x}_i)$: $\pi_1(\mathbf{x}), \dots, \pi_m(\mathbf{x})$

such that: $\rho(\mathbf{x}, \mathbf{x}_{\pi_i(\mathbf{x})}) \leq \rho(\mathbf{x}, \mathbf{x}_{\pi_{i+1}(\mathbf{x})})$

Notice: In compare to the Parameter Predictor, we are Always using the dataset, so we are not constructing a predictor function but i consult everytime the dataset

k-NN

input: a training sample $S = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$

output: for every point $\mathbf{x} \in \mathcal{X}$,

return the majority label among $\{y_{\pi_i(\mathbf{x})} : i \leq k\}$

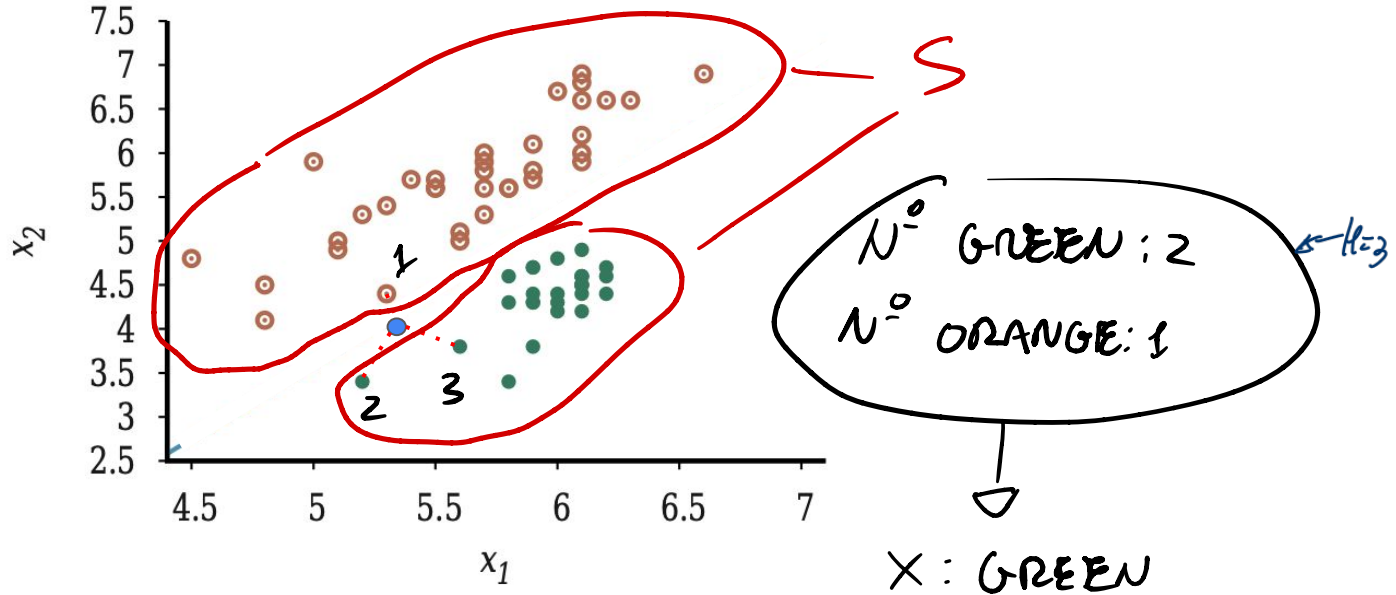
i pick the first k elements of the ordered S

kNN Binary Classification

E.g.: $k = 3$

if $k=1$ simply
choose orange

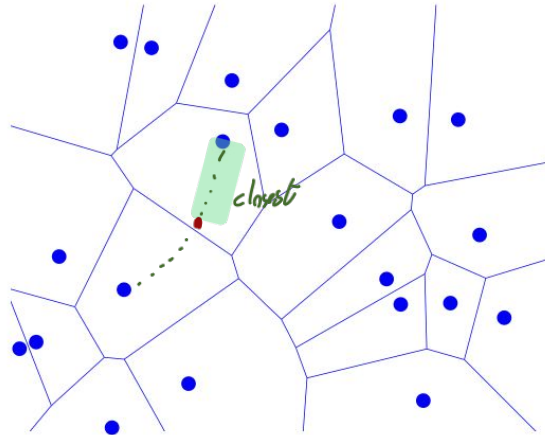
OBVIOUSLY, I COULD TAKE ALL INSTANCES
IN S AND COMPUTE THE DISTANCE FROM x
 $\forall x_i$. BUT HERE, YOU PICK THE 1st k OF
THEM, AND CLASS MAJORITY WILL WIN



kNN Binary Classification

For $k=1$, induced partition of instance space (Voronoi Tessellation):

IN THE PREVIOUS EXAMPLE,
X: ORANGE IF $k=1$ (NEAREST
CLASS TO X BASED ON S)



The blue dots are the points
of your database.
If the new point that
I want to predict belongs
in a specific region, the area
is predicted easily

Not only Binary Classification

kNN Classification in General

- Approach can be extended to multi-class
 - Just select most common class among those of kNNs

- Distance function $\rho(\mathbf{x}, \mathbf{x}_i)$ need not be Euclidean
 - Hamming distance for categorical features
 - Manhattan Distance
 - ...

it can be, but in general depends by the Domain that we are dealing with

$$\rho(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^d |x_i - x'_i|$$

kNN Classification, importance of feature scale

Feature Scale (after achieved through normalization) is a data preprocessing step used to adjust the numerical ranges of independent variables (Features) in your dataset so that they all exist on a similar scale.

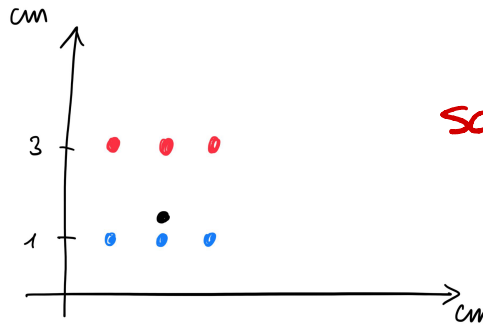
For example, if you have a dataset describing people, an Feature might be "Age" (ranging from 0 to 100) while another might be "Annual Income" (ranging from 20,000\$ to 150,000\$). Without scaling, the "Income" Feature has a mathematically much larger scale than the "Age".

EUCLIDEAN DISTANCE

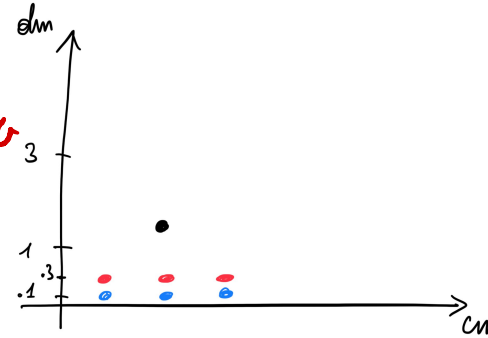
$$P(x_i, x_j) = \sqrt{\sum_{i=1}^n |x_i - x_j|^2}$$

- Approach can be extended to multi-class
 - Feature scale is critical
 - Larger features tend to be more important
 - How do age and weight compare to each other?

↳ FEATURE SCALING (NORMALIZATION)



SCALING
—D



kNN Classification, Normalization

IT'S GENERALLY USED
(NOT ONLY IN kNN)

EXAMPLE

[1, 2, 4, 5, 7, 10]

DIVIDE BY 10

[0.1, 0.2, 0.4, 0.5, 0.7, 1]

Feature scale impact (partly) mitigated by data normalization

- Z-score normalization:

- Mean of normalized data: 0
- Variance of normalized data: 1
- σ in formula is standard deviation

$$z = \frac{x - \mu}{\sigma}$$

μ ← Mean
 σ ← standard deviation

- Linear scaling normalization:

- Range is $[a, b]$

NORMALIZATION IN RANGE $[a, b]$

$$x' = a + \frac{(x - \min(x))(b - a)}{\max(x) - \min(x)}$$

Allows to normalize the data in a range $[a, b]$, for example $[0, 1]$, and dataset $[1, 3, 7, 6, 22, 15]$, i choose $x=6$: $\frac{6-1}{22-1} = \frac{5}{21}$

MAX WILL BE THE MAX, MIN WILL BE THE MIN ($a=0, b=1$)

- Both transformations are lossless (i.e., they are invertible)

THIS APPROACH IS USEFUL IF YOU KNOW THAT YOUR ALGORITHM WORKS WELL WITH NORMALIZED DATA, WHERE YOU CENTRE YOUR DATA INSTEAD OF HAVING A BIG SCALE

Normalization, in general

Data Normalization is **always recommended**, to avoid

- ^{most important} **Undesired differences in feature importance**
- Working with extremely large or small values which require large approximations
- Which one depends on data and algorithms (e.g., min-max is a good choice for Neural Networks)

kNN, Generalization Bound (HOW WELL DOES KNN PERFORM)

Binary Classification

- What can we expect of kNN wrt Generalization? Consider the 1-NN rule $k=1$

NORMALIZED POINTS IN $[0, 1]$

$h_S(x): X \rightarrow \{0, 1\}$

THEOREM 19.3 Let $\mathcal{X} = [0, 1]^d$, $\mathcal{Y} = \{0, 1\}$, and $\mathcal{D}$ be a distribution over $\mathcal{X} \times \mathcal{Y}$ for which the conditional probability function, η , is a c -Lipschitz function. Let h_S denote the result of applying the 1-NN rule to a sample $S \sim \mathcal{D}^m$. Then,

$\star$ EXPECTED ERROR

EXPECTED LOSS

FOR A RANDOM SET OF VARIABLES CLASSIFIED WITH KNN-RULE

$$\mathbb{E}_{S \sim \mathcal{D}^m} [L_{\mathcal{D}}(h_S)] \leq 2[L_{\mathcal{D}}(h^*)] + 4c\sqrt{d}m^{-\frac{1}{d+1}} \leq \epsilon$$

d : DATA DIMENSION
 m : N° OF SAMPLES IN THE DATASET
 c : CONSTANT

KNN HYPOTHESIS FUNCTION $(h_S(x): X \rightarrow \{0, 1\})$

ON BINARY CLASSIFICATION

$h^*(x) = \mathbb{1}_{[\eta(x) > 1/2]}$: Bayesian Optimal Classifier (THE BEST A STANDARD CAN DO)

Conditional Probability

$\eta(x) = \mathbb{P}[y = 1|x]$: probability that point x has label 1 PROBABILITY DISTRIBUTION

c-Lipschitz: for all $x, x' \in \mathcal{X}$, $|\eta(x) - \eta(x')| \leq c \|x - x'\|$

ACTUALLY, ϵ CAN BE REDUCED, BECAUSE IT IS BASED ON YOUR DATASET AND YOUR DATA DIMENSION

IN GENERAL, YOU DON'T KNOW THIS NUMBER

Meaning: Even if:

$$E_{S \sim D^m} [L_D(h_S)] = Z L_D(h^*) + 4c \sqrt{d} m^{-\frac{1}{d+2}}$$

in this case the expected loss can be in the worst case scenario the twice of expected loss of the Bayes Optimal Classifier.

The second thing: $4c \sqrt{d} m^{-\frac{1}{d+2}}$, depends on the dimension "d" and the number of samples "m".

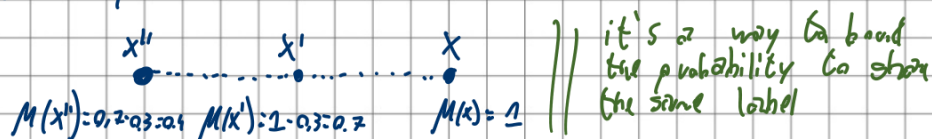
so we can mark on that

L-LIPSCHITZ:

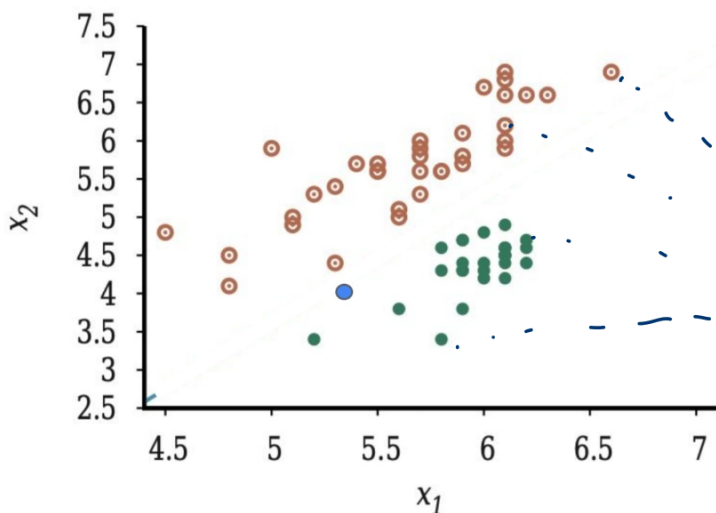
$$\text{for all } x, x' \in X, |M(x) - M(x')| \leq c \|x - x'\|$$

A function is c-Lipschitz if the difference of the probability having the same label grows linearly (c value) with the distance of the 2 points x, x' .

let's say:



let's consider an example of why is important



we can still apply it
but the point is so far
that the information is
useless!!

so the Lipschitzness
gives the idea of the fact
that the point should
be near to the other.

The curse of Dimensionality

CURSCMIT8 IS USEFUL TO SAY THAT, FOR A GIVEN x , THE CONSTANT YOU ARE FROM S , THE WORST K-NN IS; THE NEAREST, THE BETTER

kNN assumes that close-by points have same labels

When points aren't close-by, informativeness drops

In order to work well, dataset must be dense over instance space

Observe: $\mathbb{E}_{S \sim \mathcal{D}^m} [L_{\mathcal{D}}(h_S)] \leq 2 L_{\mathcal{D}}(h^*) + 4c\sqrt{d}m^{-\frac{1}{d+1}}$ $\rightarrow$ let's consider just $\geq 4c\sqrt{d}m^{-\frac{1}{d+1}}$

To have classification error smaller than ϵ wrt Bayesian Classifier, must have:

$$m \geq (4c\sqrt{d}/\epsilon)^{d+1}$$

That is, dataset size must grow exponentially wrt dimension!!!

$\rightarrow$ so d is big we practically won't have enough points to increase the dataset

SO, IN 1-NN RULE THROUGH, YOU HAVE TO ASSURE THAT YOU WILL HAVE ALWAYS AN ADDITIONAL ERROR GIVEN BY d AND m SETTINGS

kNN, Complexity

in linear regression
just 2 values.

- kNN May have a large memory requirement (needs to store entire dataset)
- What about *inference time*?
 - Need to compute distance of unseen point x wrt to all other m , thus $O(dm)$
 - Can be extremely expensive
 - OK with small datasets and small data dimensionality d
- For large datasets, optimizations can be applied, such as *k-d trees*, which facilitate searching the nearest neighbors
 - Polynomial wrt d , logarithmic wrt m
- But *k-d trees* require size exponential in d so, infeasible for large d
- Approximate approaches can be used to find "not too distant neighbours"

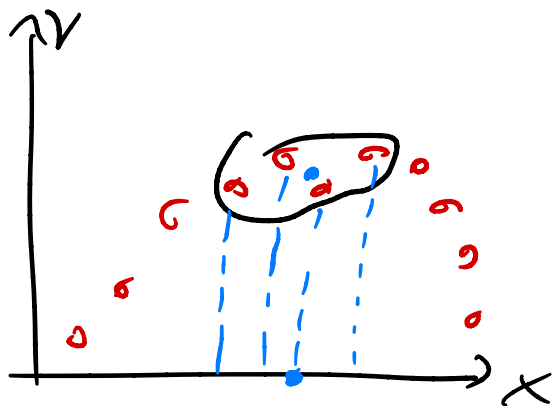
$O(d \log(m))$

kNN Regression

- kNN easily generalizes to regression:

$$h_S(\mathbf{x}) = \frac{1}{k} \sum_{i=1}^k y_{\pi_i(\mathbf{x})}$$

JUST COMPUTING AN AVERAGE INSTEAD OF MAJORITY



Parameters?

kNN still needs parameter tuning:

- What is best k ? *if k is too large (as big as the entire dataset) basically you are doing an average of everything*
- How to find it?
- Empirical approach, typically cross-validation
 - Run several test with
 - Different values of k and splitting dataset differently into training and validation set
 - Select k which led to best results in most runs

simply cost